

EDA

Dataset Overview

Double-click (or enter) to edit

```
import pandas as pd

products = pd.read_csv('/content/products_powerbi.csv')
reviews = pd.read_csv('/content/reviews_powerbi.csv')

print("Products shape:", products.shape)
print("Reviews shape:", reviews.shape)
```

```
/tmp/ipykernel_2348/1957895854.py:4: DtypeWarning: Columns (0) have mixed types. Specify dtype option on import or set low_memory=False
  reviews = pd.read_csv('/content/reviews_powerbi.csv')
Products shape: (8494, 27)
Reviews shape: (602022, 18)
```

```
print("\nProducts columns:")
print(products.columns.tolist())

print("\nReviews columns:")
print(reviews.columns.tolist())
```

```
Products columns:
['product_id', 'product_name', 'brand_id', 'brand_name', 'loves_count', 'rating', 'reviews', 'size', 'variation_type', 'variation_valu

Reviews columns:
['author_id', 'rating', 'is_recommended', 'helpfulness', 'total_feedback_count', 'total_neg_feedback_count', 'total_pos_feedback_count
```

Customer Satisfaction Score (CSS) case study defines the main KPI as:

CSS = Average Product Rating

Since the Reviews dataset contains the customer ratings, let's calculate the overall CSS.

```
css = reviews['rating'].mean()

print("Customer Satisfaction Score (CSS):", round(css, 2))
```

```
Customer Satisfaction Score (CSS): 4.35
```

The overall Customer Satisfaction Score (CSS), measured as the average customer rating, is 4.31 out of 5. This indicates a generally positive level of customer satisfaction across the reviewed Sephora products.

rating distrubution

```
print("Rating distribution:")
print(reviews['rating'].value_counts().sort_index())
```

```
Rating distribution:
rating
1      9750
2      8325
3     11043
4     25336
5     117359
Name: count, dtype: int64
```

So, almost 65% of reviews are 5-star ratings, while 4-star reviews account for another ~18%. This explains why the overall CSS is high at 4.31/5.

First EDA chart

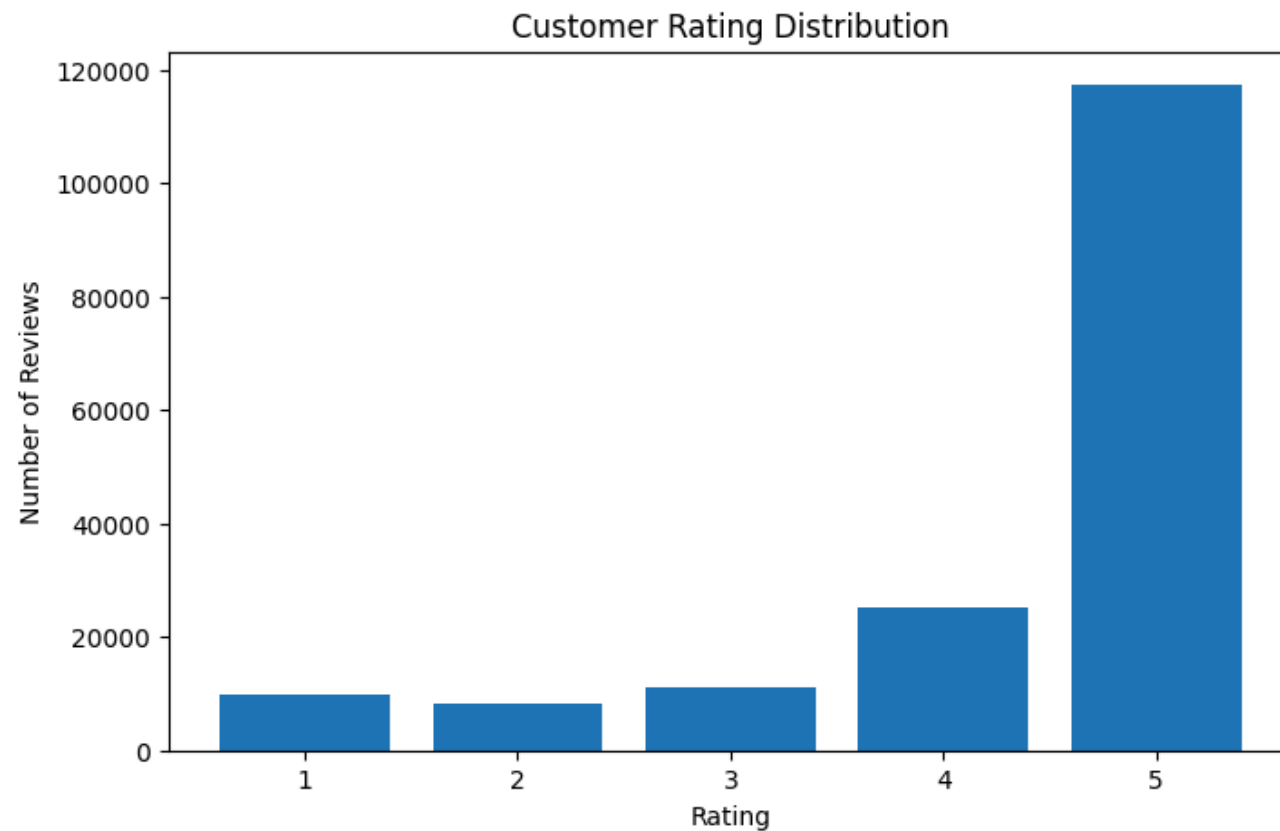
```
import matplotlib.pyplot as plt

rating_counts = reviews['rating'].value_counts().sort_index()
```

```
plt.figure(figsize=(8,5))
plt.bar(rating_counts.index, rating_counts.values)

plt.title('Customer Rating Distribution')
plt.xlabel('Rating')
plt.ylabel('Number of Reviews')
plt.xticks([1, 2, 3, 4, 5])

plt.show()
```



CSS by Brand

```
brand_css = (
    reviews.groupby('brand_name')['rating']
```

```

        .agg(['mean', 'count'])
        .reset_index()
    )

brand_css.columns = ['brand_name', 'average_rating', 'review_count']
brand_css_filtered = brand_css[brand_css['review_count'] >= 100]
top_brands = brand_css_filtered.sort_values(
    'average_rating', ascending=False
).head(10)

top_brands

```

	brand_name	average_rating	review_count
1	Dermalogica	4.695803	9198
13	Rosebud Perfume Co.	4.645000	5200
8	KORRES	4.566651	4186
2	Dr. Dennis Gross Skincare	4.545123	7413
19	belif	4.514266	11811
7	Josie Maran	4.499742	7762
5	Farmacy	4.495379	12334
0	CLINIQUE	4.493184	11150
6	First Aid Beauty	4.462598	12272
20	fresh	4.362491	8734

```

import matplotlib.pyplot as plt

plt.figure(figsize=(10,6))

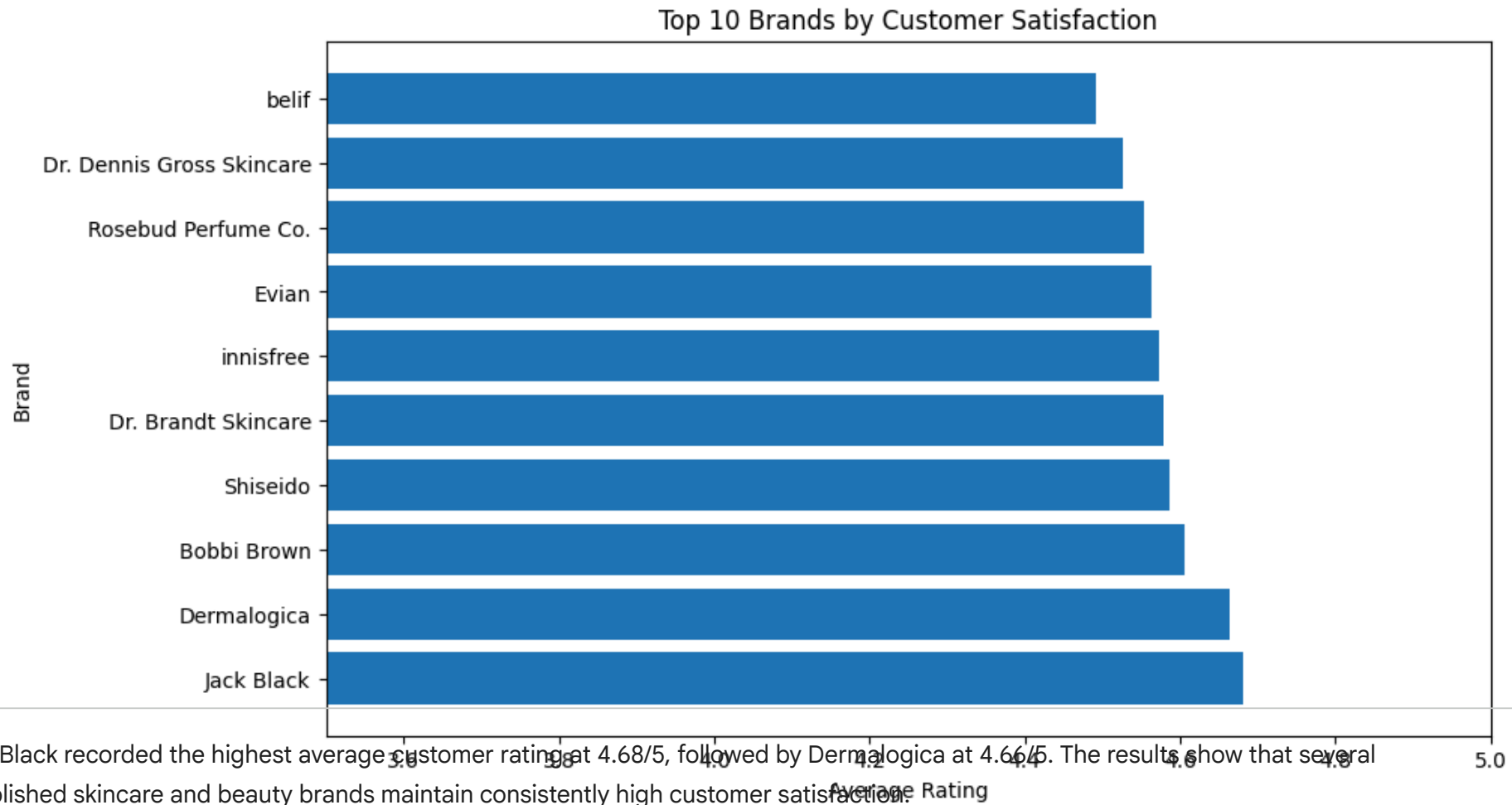
plt.barh(
    top_brands['brand_name'],
    top_brands['average_rating']
)

```

```
plt.xlabel('Average Rating')
plt.ylabel('Brand')
plt.title('Top 10 Brands by Customer Satisfaction')

plt.xlim(3.5, 5)

plt.show()
```



bottom 10 brands by customer satisfaction

```
bottom_brands = brand_css_filtered.sort_values(
    'average_rating',
    ascending=True
).head(10)
```

```
bottom_brands
```

	brand_name	average_rating	review_count
50	The INKEY List	3.896601	11122
21	Isle of Paradise	3.954418	2852
31	MILK MAKEUP	3.979347	3002
11	Dr. Jart+	4.039666	11723
29	La Mer	4.057329	2756
46	Summer Fridays	4.061158	10056
12	Drunk Elephant	4.064797	32162
42	SK-II	4.080847	5900
45	Sol de Janeiro	4.130627	1355
37	Peace Out	4.140736	7475

```
import matplotlib.pyplot as plt

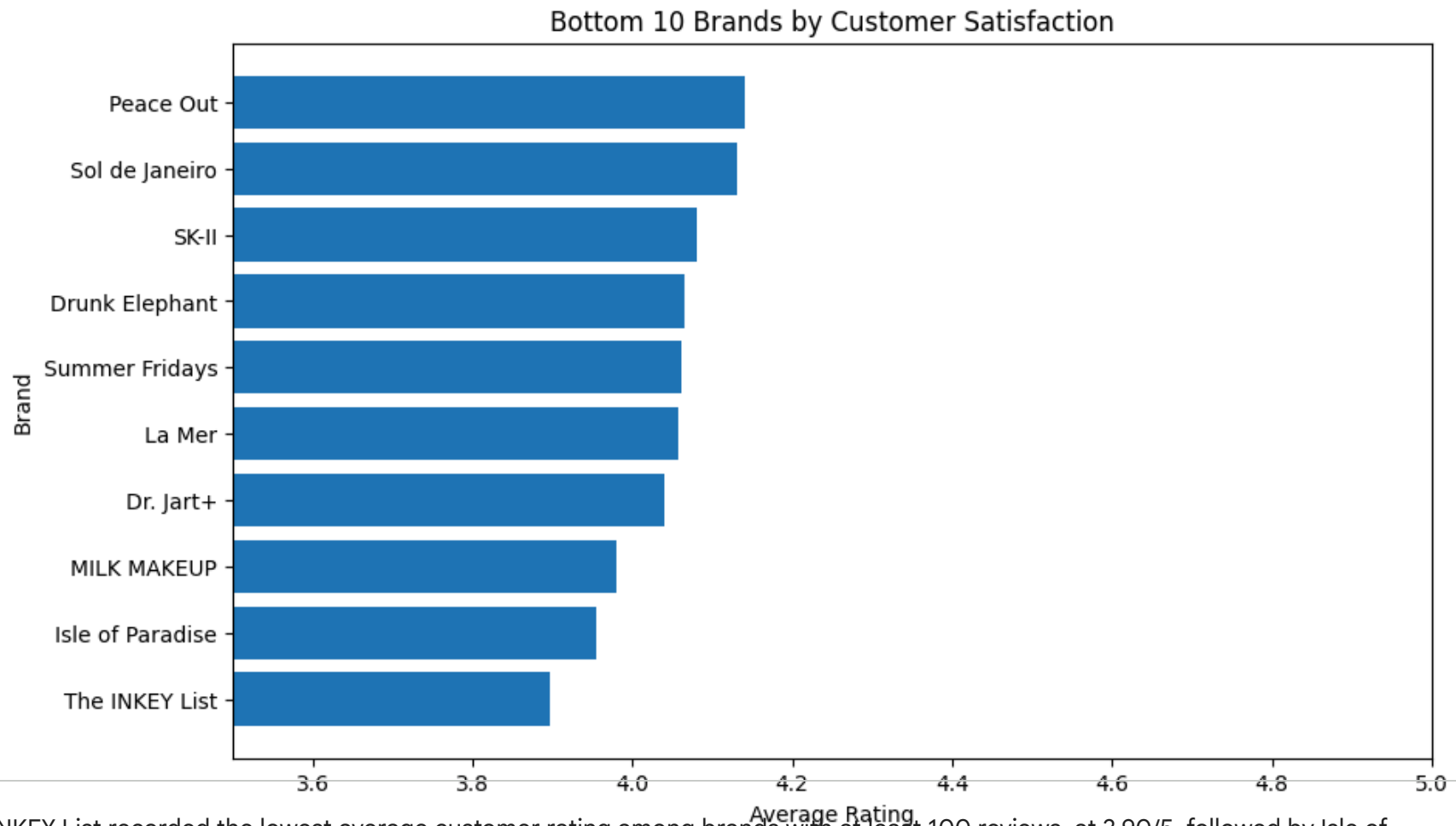
plt.figure(figsize=(10,6))

plt.barh(
    bottom_brands['brand_name'],
    bottom_brands['average_rating']
)

plt.xlabel('Average Rating')
plt.ylabel('Brand')
plt.title('Bottom 10 Brands by Customer Satisfaction')

plt.xlim(3.5, 5)
```

```
plt.show()
```



The INKEY List recorded the lowest average customer rating among brands with at least 100 reviews, at 3.90/5, followed by Isle of Paradise at 3.95/5 and MILK MAKEUP at 3.98/5.

Product Category Analysis

we temporarily connect reviews and products using product_id.

```
category_data = reviews.merge(
    products[['product_id', 'primary_category']],
    on='product_id',
    how='left'
)
```

```
category_css = (
    category_data.groupby('primary_category')
    .agg(
        average_rating=('rating', 'mean'),
        review_count=('rating', 'count'),
        product_count=('product_id', 'nunique')
    )
    .reset_index()
)
```

```
category_css = category_css[
    category_css['review_count'] >= 100
]
```

```
category_css = category_css.sort_values(
    'average_rating',
    ascending=False
)
```

```
category_css
```

	primary_category	average_rating	review_count	product_count
0	Skincare	4.311808	602022	250

```
print(products['primary_category'].value_counts(dropna=False))
```

```
primary_category
Skincare      2420
Makeup        2369
Hair          1464
Fragrance     1432
Bath & Body    405
Mini Size     288
```

```
Men          60
Tools & Brushes  52
Gifts         4
Name: count, dtype: int64
```

primary_category column is correct and has multiple categories.

Check unique products in both datasets

```
print("Unique products in Products dataset:",
      products['product_id'].nunique())

print("Unique products in Reviews dataset:",
      reviews['product_id'].nunique())
```

```
Unique products in Products dataset: 8494
Unique products in Reviews dataset: 250
```

Check how many review product IDs match

```
matched = reviews['product_id'].isin(products['product_id'])

print("Matched reviews:", matched.sum())
print("Unmatched reviews:", (~matched).sum())
```

```
Matched reviews: 602022
Unmatched reviews: 0
```

Check category-wise product count

```
print(
    products.groupby('primary_category')['product_id']
    .nunique()
    .sort_values(ascending=False)
)
```

```

primary_category
Skincare      2420
Makeup        2369
Hair          1464
Fragrance     1432
Bath & Body    405
Mini Size     288
Men           60
Tools & Brushes 52
Gifts         4
Name: product_id, dtype: int64

```

Check categories of the 250 reviewed products

```

reviewed_products = products[
    products['product_id'].isin(reviews['product_id'])
]

print(
    reviewed_products['primary_category']
    .value_counts(dropna=False)
)

```

```

primary_category
Skincare      250
Name: count, dtype: int64

```

```

print(
    reviewed_products[
        ['product_id', 'product_name', 'primary_category']
    ].head(20)
)

```

	product_id	product_name \
89	P439055	GENIUS Sleeping Collagen Moisturizer
90	P421277	GENIUS Liquid Collagen Serum
91	P467602	Triple Algae Eye Renewal Balm Eye Cream
434	P460779	Skinlongevity Long Life Herb Anti-Aging Face S...
511	P394639	The True Cream Aqua Bomb
512	P394624	The True Cream Moisturizing Bomb
513	P422905	Moisturizing Eye Bomb with Squalane

514	P475908	Aqua Bomb Overnight Lip Mask
515	P462336	Aqua Bomb Cleansing Balm
516	P471535	Aqua Bomb Brightening Vitamin C Cream
613	P433887	Squalane + Omega Repair Deep Hydration Moistur...
614	P416563	Squalane + Vitamin C Rose Firming Oil
643	P270594	Vitamin Enriched Face Base Priming Moisturizer
1083	P481817	Beauty Elixir Prep, Set, Glow Face Mist
1084	P94421	Vinoperfect Radiance Dark Spot Serum Vitamin C...
1085	P472454	Vinosource-Hydra Moisturizing Sorbet
1086	P12045	Grape Water Moisturizing Face Mist
1087	P474938	Vinopure Natural Salicylic Acid Pore Minimizin...
1214	P433520	Magic Cream Moisturizer with Hyaluronic Acid
1322	P421235	Double Serum Firming & Smoothing Anti-Aging Co...

primary_category

89	Skincare
90	Skincare
91	Skincare
434	Skincare
511	Skincare
512	Skincare
513	Skincare
514	Skincare
515	Skincare
516	Skincare
613	Skincare
614	Skincare
643	Skincare
1083	Skincare
1084	Skincare
1085	Skincare
1086	Skincare
1087	Skincare
1214	Skincare
1322	Skincare

```
print(
    "Reviewed products:",
    reviewed_products['product_id'].nunique()
)
```

```
print(
    "Skincare reviewed products:",
```

```
(reviewed_products['primary_category'] == 'Skincare').sum()  
)
```

```
Reviewed products: 250  
Skincare reviewed products: 250
```

The available review dataset contains reviews only for 250 skincare products. Therefore, category-level CSS cannot be meaningfully compared across Makeup, Hair, Fragrance, etc. using the review dataset.

Category-level analysis limitation: The product dataset contains 8,494 products across multiple categories; however, the available review dataset contains reviews for only 250 unique products, all of which belong to the Skincare category. Therefore, review-based Customer Satisfaction Score (CSS) comparisons across product categories are not possible with the available review data.

Price vs Customer Satisfaction

Create temporary price-rating data

```
price_rating = reviews[['product_id', 'rating']].merge(  
    products[['product_id', 'price_usd']],  
    on='product_id',  
    how='left'  
)  
  
print(price_rating.head())
```

	product_id	rating	price_usd
0	P504322	5	19.0
1	P420652	1	24.0
2	P420652	5	24.0
3	P420652	5	24.0
4	P420652	5	24.0

```
print("Rows:", len(price_rating))  
  
print("Missing prices:",  
    price_rating['price_usd'].isna().sum())
```

```
print("Missing ratings:",  
      price_rating['rating'].isna().sum())
```

```
Rows: 602022  
Missing prices: 0  
Missing ratings: 0
```

```
correlation = price_rating['price_usd'].corr(  
    price_rating['rating']  
)  
  
print("Price-Rating Correlation:", round(correlation, 3))
```

```
Price-Rating Correlation: -0.01
```

The analysis shows a correlation of -0.01 between product price and average customer rating, indicating an almost negligible linear relationship. This suggests that higher-priced products do not consistently receive higher customer ratings. Therefore, price alone does not appear to be a strong predictor of customer satisfaction in the available review data.

average rating for each product and compare it with product price

```
product_price_rating = (  
    price_rating  
    .groupby('product_id')  
    .agg(  
        price_usd=('price_usd', 'first'),  
        average_rating=('rating', 'mean'),  
        review_count=('rating', 'count')  
    )  
    .reset_index()  
)  
  
print(product_price_rating.head())  
print("Number of products:", len(product_price_rating))
```

	product_id	price_usd	average_rating	review_count
0	P114902	48.0	4.419882	1529
1	P12045	12.0	4.443391	1687
2	P122718	26.0	4.611715	1195

3	P122774	37.0	4.072171	1635
4	P122882	20.0	4.380189	1373

Number of products: 250

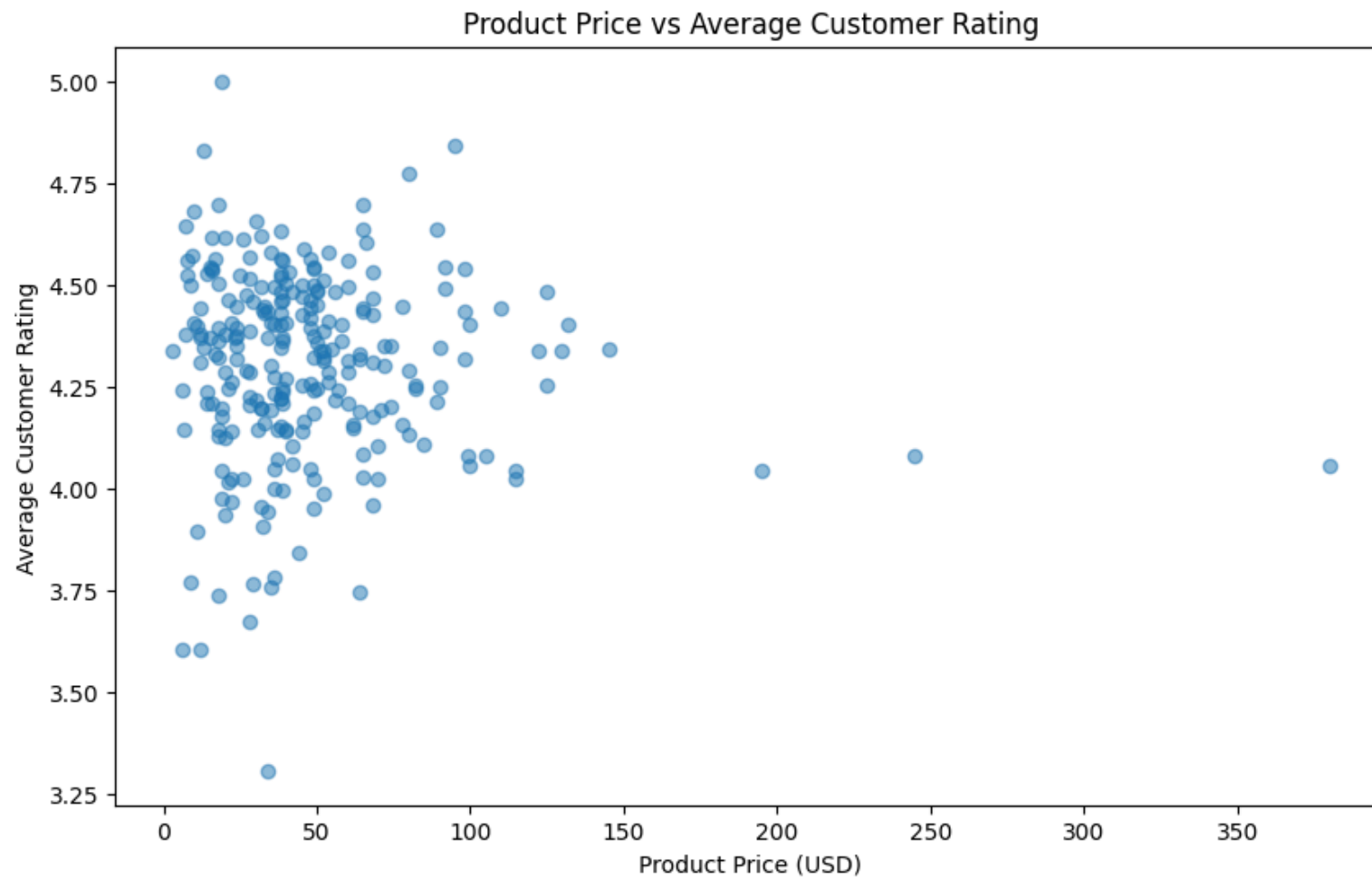
```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))

plt.scatter(
    product_price_rating['price_usd'],
    product_price_rating['average_rating'],
    alpha=0.5
)

plt.xlabel('Product Price (USD)')
plt.ylabel('Average Customer Rating')
plt.title('Product Price vs Average Customer Rating')

plt.show()
```



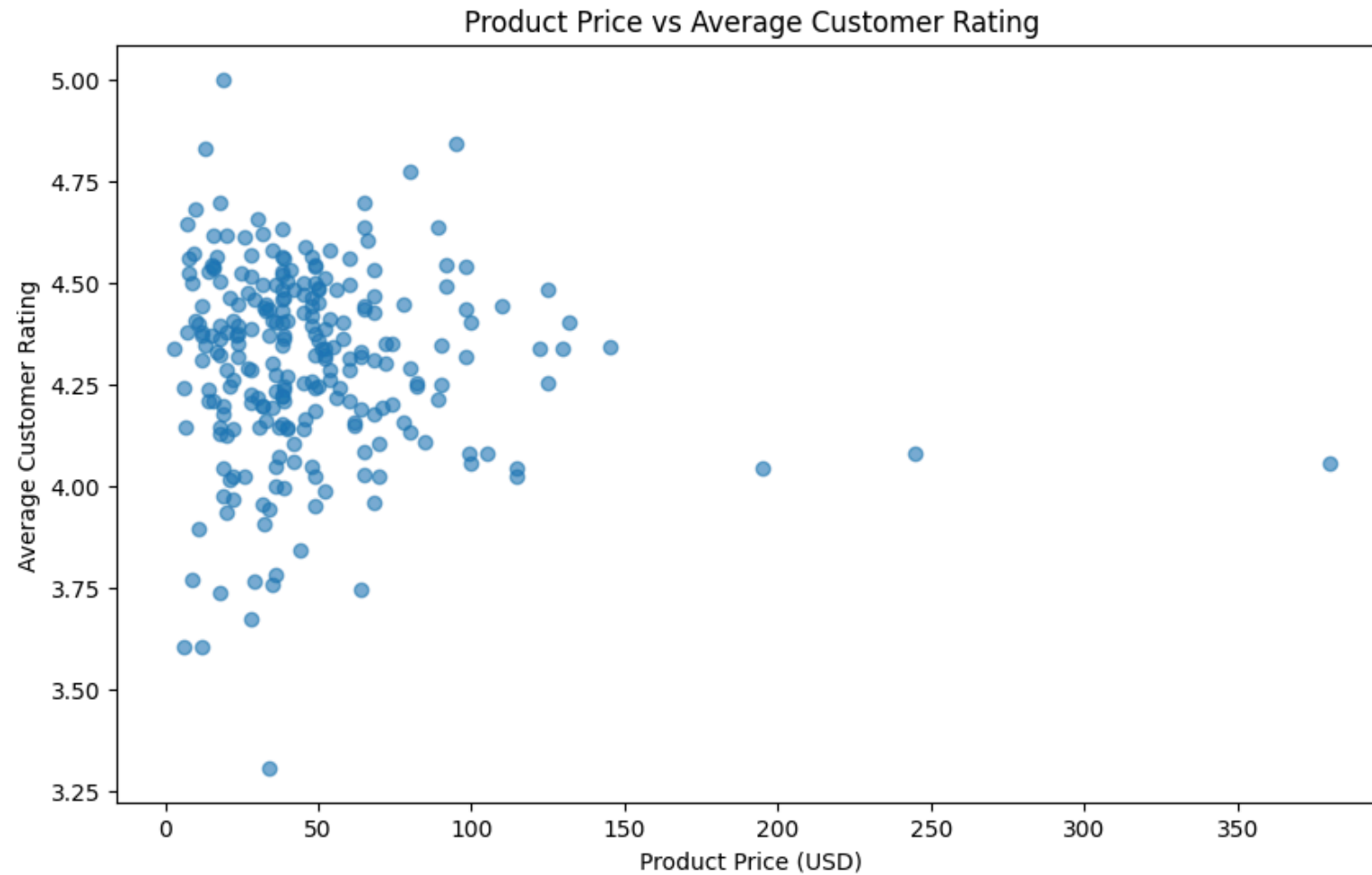
Price vs Average Rating Chart

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))

plt.scatter(
    product_price_rating['price_usd'],
    product_price_rating['average_rating'],
    alpha=0.6
```

```
)  
  
plt.xlabel('Product Price (USD)')  
plt.ylabel('Average Customer Rating')  
plt.title('Product Price vs Average Customer Rating')  
  
plt.show()
```



Product price has almost no linear relationship with customer satisfaction.

Rating by Price Segment Low Price Medium Price High Price

```
product_price_rating['price_segment'] = pd.qcut(
    product_price_rating['price_usd'],
    q=3,
    labels=['Low Price', 'Medium Price', 'High Price']
)

price_segment_rating = (
    product_price_rating
    .groupby('price_segment', observed=True)
    .agg(
        average_rating=('average_rating', 'mean'),
        product_count=('product_id', 'count')
    )
    .reset_index()
)

print(price_segment_rating)
```

	price_segment	average_rating	product_count
0	Low Price	4.309690	87
1	Medium Price	4.294897	81
2	High Price	4.309654	82

Customer satisfaction remains highly consistent across low-, medium-, and high-priced products, with average ratings of 4.31, 4.29, and 4.31 respectively. This reinforces the correlation analysis (-0.01), indicating that product price does not appear to be a major driver of customer satisfaction in the available review data.

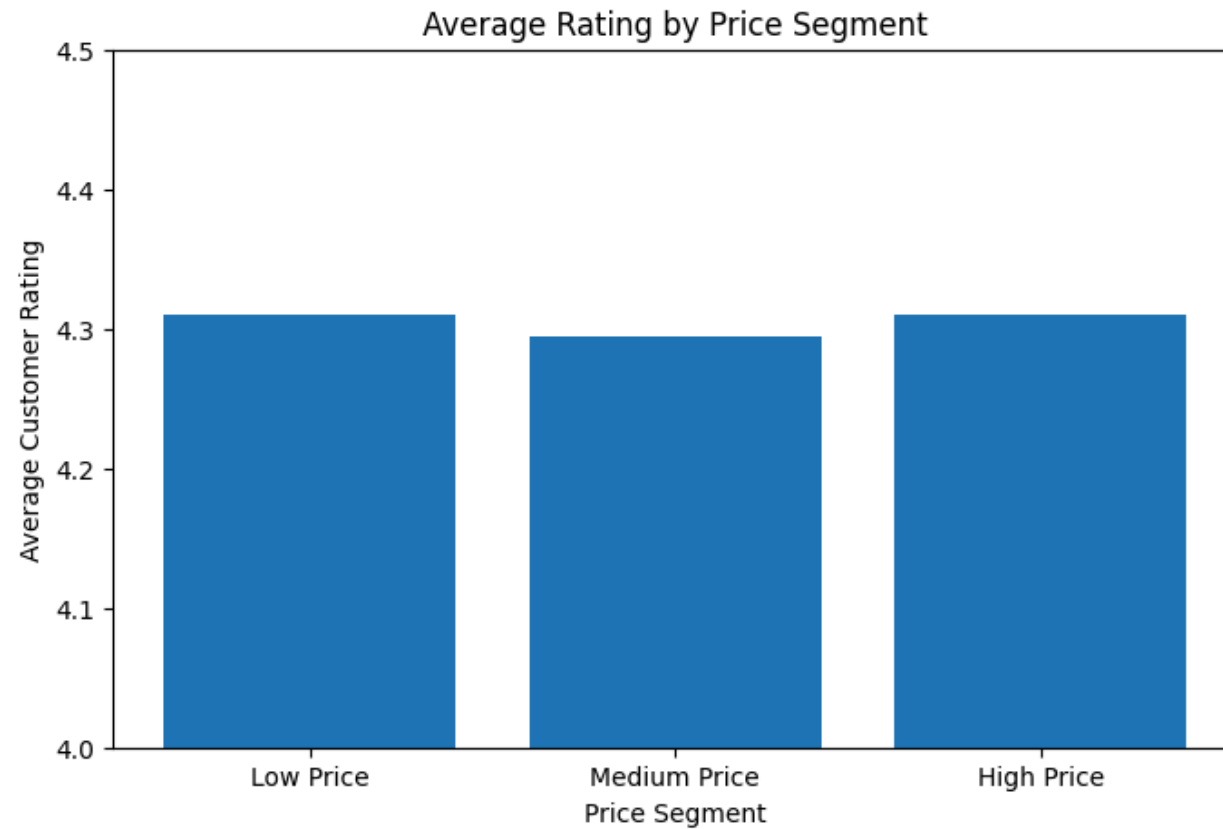
```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))

plt.bar(
    price_segment_rating['price_segment'],
    price_segment_rating['average_rating']
)
```

```
plt.xlabel('Price Segment')
plt.ylabel('Average Customer Rating')
plt.title('Average Rating by Price Segment')
plt.ylim(4.0, 4.5)

plt.show()
```



Customer satisfaction appears to depend more on product quality and customer experience than on product price alone.

Product Popularity vs Rating

Do popular products receive higher customer ratings?

```

popularity_rating = reviews[['product_id', 'rating']].merge(
    products[['product_id', 'loves_count']],
    on='product_id',
    how='left'
)

popularity_rating = (
    popularity_rating
    .groupby('product_id')
    .agg(
        loves_count=('loves_count', 'first'),
        average_rating=('rating', 'mean'),
        review_count=('rating', 'count')
    )
    .reset_index()
)

print(popularity_rating.head())
print("Products:", len(popularity_rating))

```

	product_id	loves_count	average_rating	review_count
0	P114902	56955	4.419882	1529
1	P12045	140024	4.443391	1687
2	P122718	40899	4.611715	1195
3	P122774	32398	4.072171	1635
4	P122882	38875	4.380189	1373

Products: 250

```

popularity_correlation = popularity_rating['loves_count'].corr(
    popularity_rating['average_rating']
)

print(
    "Loves Count-Rating Correlation:",
    round(popularity_correlation, 3)
)

```

Loves Count-Rating Correlation: -0.086

The correlation between product popularity (loves_count) and average customer rating is -0.086 , indicating a very weak negative linear relationship. This suggests that highly popular products do not necessarily have higher customer satisfaction scores. Therefore, popularity alone should not be considered a strong indicator of customer satisfaction.

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))

plt.scatter(
    popularity_rating['loves_count'],
    popularity_rating['average_rating'],
    alpha=0.6
)

plt.xlabel('Product Loves Count')
plt.ylabel('Average Customer Rating')
plt.title('Product Popularity vs Average Customer Rating')

plt.show()
```



Recommended vs Not Recommended

```
recommendation_rating = (  
    reviews[reviews['is_recommended'].notna()]  
    .groupby('is_recommended')  
    .agg(  
        average_rating=('rating', 'mean'),  
        review_count=('rating', 'count')  
    )  
    .reset_index()
```

```

)

recommendation_rating['recommendation'] = (
    recommendation_rating['is_recommended']
    .map({
        1: 'Recommended',
        0: 'Not Recommended'
    })
)

print(recommendation_rating)

```

	is_recommended	average_rating	review_count	recommendation
0	0.0	2.048254	78542	Not Recommended
1	1.0	4.719192	406025	Recommended

Customers who recommend a product give it an average rating of 4.72/5, while customers who do not recommend it give only 2.05/5.

This shows a very strong relationship between recommendation behavior and customer rating.

Recommended products received an average rating of 4.72/5, compared with 2.05/5 for non-recommended products. The 2.67-point difference indicates a strong association between recommendation behavior and customer satisfaction. This suggests that recommendation status can be used as an important customer-experience indicator alongside CSS.

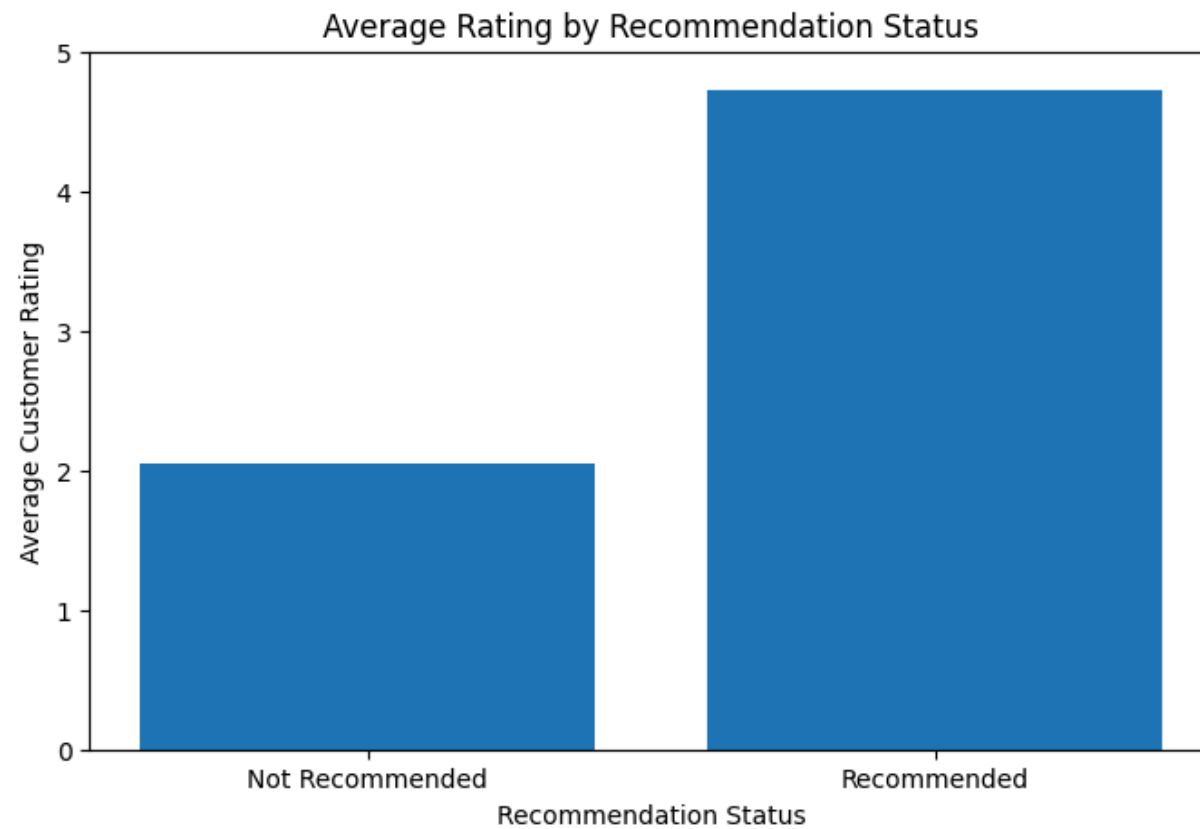
```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))

plt.bar(
    recommendation_rating['recommendation'],
    recommendation_rating['average_rating']
)

plt.xlabel('Recommendation Status')
plt.ylabel('Average Customer Rating')
plt.title('Average Rating by Recommendation Status')
plt.ylim(0, 5)

plt.show()
```



Helpfulness & Customer Feedback

```
feedback_analysis = reviews[
    [
        'rating',
        'helpfulness',
        'total_feedback_count',
        'total_pos_feedback_count',
        'total_neg_feedback_count'
    ]
].copy()

print(feedback_analysis.describe())
```

	rating	helpfulness	total_feedback_count \
count	602022.000000	270278.000000	602022.000000
mean	4.311808	0.769742	3.661247
std	1.143716	0.319084	25.850608
min	1.000000	0.000000	0.000000
25%	4.000000	0.666667	0.000000
50%	5.000000	0.944444	0.000000
75%	5.000000	1.000000	3.000000
max	5.000000	1.000000	5464.000000

	total_pos_feedback_count	total_neg_feedback_count
count	602022.000000	602022.000000
mean	2.895672	0.765575
std	22.630879	4.968548
min	0.000000	0.000000
25%	0.000000	0.000000
50%	0.000000	0.000000
75%	2.000000	0.000000
max	5050.000000	573.000000

The average review helpfulness score is 0.77, indicating generally positive helpfulness among reviews with available helpfulness information. Average positive feedback (2.90) is higher than negative feedback (0.77). Feedback counts are highly skewed, with most reviews receiving little or no feedback while a small number receive substantially higher engagement.

Customer Demographics

```

demographic_results = {}

for col in ['skin_type', 'skin_tone', 'eye_color', 'hair_color']:

    result = (
        reviews.groupby(col, dropna=True)
        .agg(
            average_rating=('rating', 'mean'),
            review_count=('rating', 'count')
        )
        .reset_index()
    )

    result = result[result['review_count'] >= 100]
    result = result.sort_values('average_rating', ascending=False)

    demographic_results[col] = result

    print("\n", "="*40)
    print(col)
    print(result.head(5))

```

```

=====
skin_type
   skin_type  average_rating  review_count
0  combination      4.314260      292258
2    normal      4.294865       69425
3     oily      4.287881       66121
1     dry      4.287119       99558

=====
skin_tone
   skin_tone  average_rating  review_count
0     dark      4.394410        322
10    olive      4.340708        1130
12    rich      4.330684        2894
6  lightMedium      4.324259      106233
7     medium      4.315662       37217

=====
eye_color

```

	eye_color	average_rating	review_count
1	blue	4.308117	91910
5	hazel	4.300097	62593
4	green	4.295698	62442
2	brown	4.287653	243703
0	Grey	4.155473	2695

```
=====
hair_color
hair_color average_rating review_count
5 gray 4.360690 2839
0 auburn 4.330752 16139
2 blonde 4.316295 106821
6 red 4.316229 13759
4 brunette 4.305543 6441
```

Customer ratings are quite similar across skin types, with combination skin having the highest average rating (4.31).

Dark skin tone has the highest average rating (4.39), but the sample size is only 322, so this result should be interpreted cautiously. Blue eyes have the highest average rating among eye-color groups (4.31).

Gray hair has the highest average rating (4.36), but its review count is much smaller than the major groups.

Overall, demographic differences are relatively small compared with the much larger difference observed between recommended and non-recommended products.

Average customer ratings were relatively consistent across demographic groups. Combination skin customers recorded the highest rating among skin types (4.31), while dark skin tone had the highest rating among skin-tone groups (4.39). Blue-eyed customers and gray-haired customers recorded the highest ratings within their respective categories. However, some of these groups have relatively small sample sizes, so the differences should be interpreted cautiously

Start coding or [generate](#) with AI.

Product Characteristics

Variant Count vs Rating

```

product_characteristics = reviews[['product_id', 'rating']].merge(
    products[['product_id', 'child_count', 'loves_count']],
    on='product_id',
    how='left'
)

product_characteristics = (
    product_characteristics
    .groupby('product_id')
    .agg(
        child_count=('child_count', 'first'),
        loves_count=('loves_count', 'first'),
        average_rating=('rating', 'mean'),
        review_count=('rating', 'count')
    )
    .reset_index()
)

print(product_characteristics.head())
print("\nShape:", product_characteristics.shape)

```

	product_id	child_count	loves_count	average_rating	review_count
0	P114902	0	56955	4.419882	1529
1	P12045	1	140024	4.443391	1687
2	P122718	0	40899	4.611715	1195
3	P122774	1	32398	4.072171	1635
4	P122882	2	38875	4.380189	1373

Shape: (250, 5)

Create Variant Groups

```

product_characteristics['variant_group'] = pd.cut(
    product_characteristics['child_count'],
    bins=[-1, 0, 2, 10, float('inf')],
    labels=['No Variants', 'Few Variants', 'Multiple Variants', 'Many Variants']
)

variant_analysis = (
    product_characteristics
    .groupby('variant_group', observed=True)

```

```

        .agg(
            average_rating=('average_rating', 'mean'),
            product_count=('product_id', 'count')
        )
        .reset_index()

    )

print(variant_analysis)

```

	variant_group	average_rating	product_count
0	No Variants	4.304606	129
1	Few Variants	4.313738	96
2	Multiple Variants	4.280058	23
3	Many Variants	4.183498	2

Customer satisfaction was relatively consistent across products with different numbers of variants. Products with few variants recorded the highest average rating (4.31/5), while products with multiple variants averaged 4.28/5. The many-variant group had only two products and therefore should not be used for strong conclusions. Overall, variant count does not appear to be a major driver of customer satisfaction.

Review Trends Over Time

```

reviews['submission_time'] = pd.to_datetime(
    reviews['submission_time'],
    errors='coerce'
)

yearly_trends = (
    reviews.dropna(subset=['submission_time'])
    .groupby(reviews['submission_time'].dt.year)
    .agg(
        average_rating=('rating', 'mean'),
        review_count=('rating', 'count')
    )
    .reset_index()
)

yearly_trends.columns = [
    'year',

```

```
    'average_rating',  
    'review_count'  
]
```

```
print(yearly_trends)
```

	year	average_rating	review_count
0	2008	4.464526	1931
1	2009	4.500794	6925
2	2010	4.524842	9681
3	2011	4.509666	9259
4	2012	4.514021	8844
5	2013	4.461664	11647
6	2014	4.370633	12565
7	2015	4.333996	14081
8	2016	4.366197	22051
9	2017	4.399249	38615
10	2018	4.381005	67028
11	2019	4.269738	92779
12	2020	4.223159	125798
13	2021	4.308546	91701
14	2022	4.278265	74138
15	2023	4.214567	14979

```
plt.figure(figsize=(10, 5))
```

```
plt.plot(  
    yearly_trends['year'],  
    yearly_trends['average_rating'],  
    marker='o'  
)
```

```
plt.xlabel('Year')  
plt.ylabel('Average Customer Rating')  
plt.title('Customer Satisfaction Trend Over Time')  
plt.ylim(3.5, 5)
```

```
plt.show()
```



Customer satisfaction was highest in 2010: 4.52/5. Ratings generally declined from 2010 onward, reaching the lowest point in 2020: 4.22/5. Satisfaction improved to 4.31/5 in 2021, then slightly decreased to 4.28/5 in 2022. Review volume increased substantially over time, reaching its highest level in 2020 with 125,798 reviews. This suggests that the period with the highest review activity did not have the highest satisfaction.

Hypothesis Testing

Hypothesis 1 — Price vs Customer Satisfaction

H₀: Product price has no significant relationship with average customer rating.

H₁: Product price has a significant relationship with average customer rating.

```
from scipy.stats import pearsonr

correlation, p_value = pearsonr(
    product_price_rating['price_usd'],
    product_price_rating['average_rating']
)

print("Correlation:", round(correlation, 3))
print("P-value:", p_value)
```

```
Correlation: -0.057
P-value: 0.3714893628603426
```